New York University Tandon School of Engineering
Computer Science and Engineering

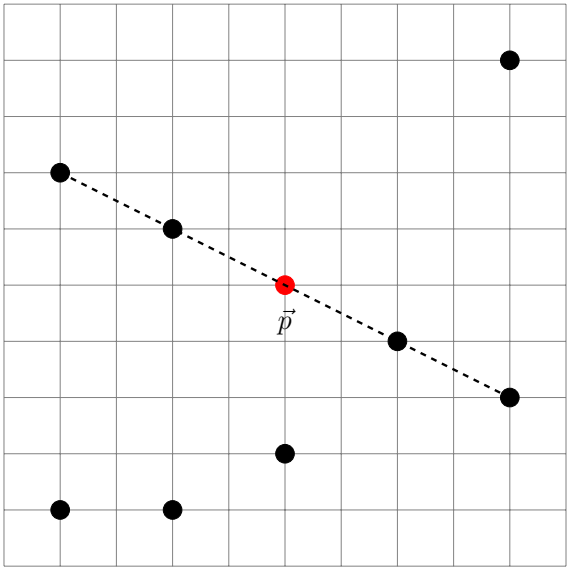
CS-GY 6033: Written Homework 3.
Due Thursday, March 05, 2026, 11:59pm.

Collaboration is allowed on this problem set, but solutions must be written-up individually. If we suspect that you copied your solution directly from AI, we will set up a meeting with you where you will need to explain your solution. If you fail to do so, you will receive zero points for the homework.

Problem 1: Collinear Points

Consider the problem of finding collinear points. Suppose you are given a set X of n points in $\mathbb{R}^2$, along with a query point $\vec{p} \in \mathbb{R}^2$. Your goal is to calculate the maximum number of points in X that fall on the same straight line containing $\vec{p}$. That is, considering all of the (infinitely) many lines containing $\vec{p}$, find the one that contains the most points from X.

For example, in the picture below, the black points are those in X and the red point represents $\vec{p}$. The maximum number of points in X that falls on a line containing $\vec{p}$ is four (those on the dashed line).



In a file named `max_points_on_line.py`, write a function `max_points_on_line(X, p)` which accepts the following arguments:

- `X`: A list of points, each point being a 2-tuple of integers.
- `p`: A query point represented as a 2-tuple of integers.

Your function should return the maximum number of points in `X` that fall on the same straight line containing $\vec{p}$. Your answer should be an integer. If `X` is empty, return `0`. If a point $\vec{x}$ in `X` is such that $\vec{x} = \vec{p}$, you should consider to be on the same line as $\vec{p}$ (even if there are no other points).

Your algorithm should take expected time that is linear in the number of points in `X`.

Example (using the points in the picture above):

```
>>> X = [
... (1, 1),
... (3, 1),
... (5, 2),
... (3, 6),
... (1, 7),
```

```
... (7, 4),
... (9, 3),
... (9, 9)
... ]
>>> p = (5, 5)
>>> max_points_on_line(X, p)
4
```

Problem 2: Adjacency Matrix Properties

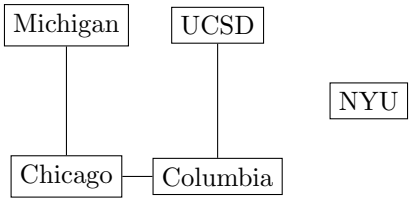
Suppose A is the adjacency matrix of a simple undirected graph.

- Let A^2 be the *squared* matrix, obtained by matrix multiplying A by itself. Show that the (i, j) entry of A^2 is the number of ways to get from i to j in exactly two hops (i.e., the number of paths of length two between node i and node j). *Hint:* Consult your linear algebra notes/textbook to remember a formula for the (i, j) entry of the product of two matrices.
- Show that $\frac{1}{6}\sum_{i=1}^{n}(A^3)_{ii}$ is the number of triangles in the graph. *Hint:* expand the diagonal entries $(A^3)_{ii}$.

  Interestingly, standard message-passing Graph Neural Networks (GNNs), a powerful Machine Learning architecture, cannot reliably count the number of triangles in a graph. See this paper [K Xu, W Hu, J Leskovec, S Jegelka. "How powerful are graph neural networks?"] for further reading.

Problem 3: Good vs. Evil

We can use a graph to represent rivalries between universities. Each node in the graph is a university, and an edge exists between two nodes if those two schools are rivals. For instance, the graph below represents the fact that Chicago and Michigan are rivals, Chicago and Columbia are rivals, UCSD and Columbia are rivals, but NYU does not have a rival.



In a file called `assign_good_and_evil.py`, write a function `assign_good_and_evil(graph)` which determines if it is possible to label each university as either "good" or "evil" such that every rivalry is between a "good" school and an "evil" school. The input to the function will be a graph represented as a Python dictionary (an adjacency list). If there is a way to label each node as "good" and "evil" so that every rivalry is between a "good" school and an "evil" school, your function should return it as a dictionary mapping each node to a string, `'good'` or `'evil'`; if such a labeling is not possible, your function should return `None`.

For example:

```
example_graph = {
"Michigan": ["Chicago"],
"Chicago": ["Michigan", "Columbia"],
"Columbia": ["Chicago", "UCSD"],
"UCSD": ["Columbia"],
"NYU": []
}

assign_good_and_evil(example_graph)
```

One possible outout is

```
    {
    "Chicago": "good",
    "Michigan": "evil",
    "Columbia": "evil",
    "UCSD": "good",
    "NYU": "good"
}
```

If in the above graph, there is also an edge between `Michigan' and `Columbia', then there does not exist such a label and your funtcion should return None.

Of course, there might be several ways to label the graph – your code need only return one labeling.

Problem 4: Network Latency Analysis

You are working as a systems engineer for a cloud infrastructure company. The company maintains a hierarchical communication network connecting data centers across different geographic regions.

Due to regulatory and architectural constraints, the network topology forms a tree:

- Each node represents a data center.
- Each edge represents a direct communication link.
- Each link has an associated latency (communication delay).

The company wants to determine the worst-case communication delay between any two data centers. This value represents the maximum time required for data to travel between the two most distant locations in the network.

1. Design an efficient algorithm to compute the maximum communication delay between any pair of data centers, assuming each communication link has unit latency.
2. Does your algorithm still work if communication links have arbitrary (nonnegative) latencies? Either prove correctness or provide a counterexample.
3. To reduce the worst-case communication delay, the company considers adding additional communication links, thereby introducing cycles into the network.
   (a) Provide an example of a connected unweighted graph with cycles where your original algorithm fails.
   (b) Modify your algorithm so that it correctly computes the worst-case communication delay in a general unweighted connected graph.

   Interestingly, under common complexity assumptions (e.g., SETH), there is strong evidence that no truly subquadratic algorithm exists for this problem, so do not worry if your algorithm is quadratic or worst—if you find a substantially faster exact algorithm, you will likely become famous (and possibly rich).